

HapRepair: 面向 OpenHarmony ArkTS 代码缺陷修复的跨文件依赖上下文增强方法尝试

王安晟, 周正彦, 罗章烨, 寇智勇, 詹柏涵

摘要: 随着 OpenHarmony 生态的快速发展, ArkTS 代码在界面逻辑、状态管理与性能优化方面面临日益复杂的缺陷修复需求. 传统基于规则或静态分析的方法在处理多样化、上下文相关的缺陷时存在覆盖面不足、迁移成本高和人工维护负担重等问题. 近年来, 大语言模型(large language model, LLM)在代码理解与生成任务中展现出较强潜力, 但其在缺陷修复场景下仍容易受到提示信息不足、规则语义不明确以及上下文缺失等因素影响, 导致修复结果不稳定或过度依赖模型先验.

本文面向 HapRepair 项目, 设计并实现了一种将跨文件依赖分析结果集成到模型上下文中的 ArkTS 缺陷修复方法. 与直接使用 LLM 进行修复的基线方法相比, 该方法在提示词中显式引入缺陷规则名称、规则描述以及与当前缺陷相关的跨文件依赖信息, 包括依赖文件中的类型定义、接口声明、函数签名及调用链信息, 使模型能够更准确地理解缺陷类型并生成更贴近预期的修复代码. 实验结果表明, 在 266 个缺陷对构成的数据集上, 基于上下文增强的修复方法的修复率由 85.3% 提升至 93.2%, 较基线提升 7.9 个百分点, 显著降低了未修复样本的数量. 进一步地, 在多个规则类别上均观察到明显提升, 例如 typed-array-check、use-local-var-to-replace-state-var 和 multiple-associations-state-var-check 等规则的修复率分别提升了 47%、40% 和 33%.

需要说明的是, 本文所报告的优化效果是在未引入完整 RAG(retrieval-augmented generation) 检索模块的条件下测得的, 因此更准确地说, 它验证的是"跨文件依赖分析集成到模型上下文"这一提示增强策略对缺陷修复性能的提升效果, 而非完整 RAG 系统的最终性能. 该结果表明, 针对 ArkTS 这类规则型、上下文强相关的代码缺陷, 将结构化依赖上下文注入模型提示词, 能够有效补偿 LLM 在任务理解上的局限, 为后续构建更完整的 RAG 增强修复框架提供了重要的实验依据.

关键词: OpenHarmony; ArkTS; 代码修复; 大语言模型; 规则上下文; 提示工程

HapRepair: Cross-File Dependency Context Enhancement for OpenHarmony ArkTS Code Defect Repair

Wang Ansheng, Zhou Zhengyan, Luo Zhangye, Kou Zhiyong, Zhan Baihan

Abstract: With the rapid development of the OpenHarmony ecosystem, ArkTS code faces increasingly complex defect repair demands in UI logic, state management, and performance optimization. Traditional rule-based or static analysis methods suffer from limited coverage, high migration costs, and heavy manual maintenance when dealing with

diverse and context-related defects. In recent years, large language models (LLMs) have demonstrated strong potential in code understanding and generation tasks, but their application in defect repair scenarios remains susceptible to insufficient prompt information, unclear rule semantics, and missing context, leading to unstable repair results and excessive reliance on model priors.

This paper presents the HapRepair project, which designs and implements an ArkTS defect repair method that integrates cross-file dependency analysis results into the model context. Compared with baseline methods that directly use LLMs for repair, this method explicitly introduces defect rule names, rule descriptions, and cross-file dependency information related to the current defect—including type definitions, interface declarations, function signatures, and call chain information from dependent files—into the prompt, enabling the model to more accurately understand defect types and generate repair code closer to expectations. Experimental results on a dataset of 266 defect pairs show that the context-enhanced repair method improves the repair rate from 85.3% to 93.2%, a relative improvement of 7.9 percentage points, significantly reducing the number of unrepairable samples. Furthermore, notable improvements were observed across multiple rule categories, with repair rates for rules such as `typed-array-check`, `use-local-var-to-replace-state-var`, and `multiple-associations-state-var-check` increasing by 47%, 40%, and 33%, respectively.

It should be noted that the reported optimization effects were measured without introducing a complete RAG retrieval module. Therefore, more accurately, this validates the improvement effect of the "cross-file dependency analysis integrated into model context" prompt enhancement strategy on defect repair performance, rather than the final performance of a complete RAG system. The results demonstrate that for rule-based, context-dependent ArkTS code defects, injecting structured dependency context into model prompts can effectively compensate for LLM limitations in task understanding, providing important experimental evidence for subsequent construction of more complete RAG-enhanced repair frameworks.

Key words: OpenHarmony; ArkTS; code repair; large language model; rule context; prompt engineering

1 引言

OpenHarmony 是华为面向智能终端和物联网场景推出的分布式操作系统生态, 其应用开发主要依托 ArkTS 语言及其配套 UI 与系统能力模型. 随着应用复杂度不断提升, ArkTS 代码中出现了大量与性能、状态管理、循环结构、资源使用和 UI 交互相关的缺陷问题. 这些缺陷常常具有较强的"规则性"特征, 例如某些不规范的写法会导致性能下降、状态更新不一致、事件处理逻辑异常或组件结构冗余.

传统缺陷修复方法主要依赖静态规则、模板匹配或人工经验,虽然在规则明确且模式稳定的场景下表现较好,但其通用性和适应性有限^[1,2].特别是在实际工程中,代码缺陷往往分布在不同组件、不同文件甚至不同开发习惯之间,单纯依靠固定规则难以覆盖所有变体.同时,人工修复方式费时费力,不利于大规模软件演进场景下的自动化治理.

近年来,大语言模型在代码生成、代码理解和程序修复等任务中表现出显著能力^[4-6].相较于传统方法,LLM 在语义理解和自然语言对齐方面具有优势,能够根据给定上下文生成更灵活的修复建议.然而,直接将 LLM 应用于缺陷修复时,也会暴露出明显问题: (1) 模型可能无法准确识别缺陷背后的规则语义; (2) 缺少足够的领域上下文时,模型容易生成"看起来合理但并不符合工程规范"的修复; (3) 对于多规则、多上下文的修复任务,模型输出不稳定,容易出现修复不完全或修复错误的问题^[7,8].

检索增强生成(retrieval-augmented generation, RAG)技术通过在生成阶段动态引入外部知识,能够显著增强模型对任务背景和领域知识的理解^[9,10]. RAG 的优势在于可以把与当前缺陷相关的修复示例、规范说明或历史修复案例纳入提示词,从而提高生成质量.然而, RAG 的实现依赖检索系统、向量索引、知识库维护以及额外的计算开销.对于当前实验环境而言,若直接接入完整的 RAG 系统,往往需要更大规模的知识库与更复杂的工程配置.

因此,如何在不显著增加计算成本的前提下,提高 LLM 在 ArkTS 缺陷修复任务中的规则感知能力与跨文件理解能力,成为一个重要问题.基于这一背景,本文提出了一种"跨文件依赖分析集成到模型上下文"的方法,通过在提示词中显式加入缺陷规则名称、自然语言规则说明以及与当前缺陷相关的跨文件依赖上下文,使模型能够在修复阶段更充分地感知缺陷类别、修复意图与依赖关系.与单纯直接提示的修复方式相比,该方法不需要引入完整的检索模块,也能够大多数规则类型上显著提升修复表现.本文在仓库提供的实验脚本与评估逻辑基础上,对 266 个缺陷对进行系统对比实验,并从修复率、规则粒度表现与 Token 消耗三个维度完成评估.

本文的主要贡献总结如下:

- (1) 提出了一种将跨文件依赖分析结果集成到 LLM 提示词中的 ArkTS 缺陷修复方法,通过显式注入规则上下文与跨文件依赖信息,增强模型对规则型缺陷的理解能力.
- (2) 在 266 个缺陷对构成的数据集上进行了系统对比实验,验证了规则上下文增强方法相比直接修复方法在修复率上的显著提升(从 85.3% 提升至 93.2%).
- (3) 从规则粒度分析了不同类别缺陷的修复表现,发现结构性较强、语义较复杂的规则在上下文增强下收益更为显著.
- (4) 分析了上下文增强对模型调用成本的影响,证明该方法在提升效果的同时并未显著增加费用负担.

本文第 2 节介绍相关工作. 第 3 节介绍 HapRepair 系统架构与模块概览. 第 4 节详细介绍方法设计. 第 5 节描述实验设计. 第 6 节呈现实验结果与分析. 第 7 节进行进一步讨论与展望. 第 8 节总结全文.

2 相关工作

2.1 基于规则的缺陷检测与修复

传统软件缺陷修复方法通常依赖静态规则、代码规范与模式匹配^[1,2]. 对于 ArkTS 这类声明式 UI 语言而言, 许多缺陷问题具有较强的领域规则特征, 例如状态变量滥用、冗余容器、循环中重复计算、回调函数中不必要的变量声明等. 这类规则通常可以由静态分析工具或人工经验总结出来, 进而转化为可执行的检测规则. 然而, 规则驱动方法的局限主要在于: 一方面, 规则覆盖范围受限; 另一方面, 规则的维护和扩展成本较高. 对于不同项目、不同代码风格、不同版本的 ArkTS 代码, 规则之间往往需要持续更新, 难以适应不断演化的开发实践. 更重要的是, 许多缺陷并不局限于单文件, 而是依赖于组件定义、接口声明、状态变量传播以及调用链关系, 因此仅依靠局部规则无法充分表达修复语义.

2.2 基于大语言模型的代码修复

大语言模型为代码修复问题带来了新的思路. 利用自然语言指令、代码上下文和少量示例, 模型能够生成针对性修复补丁^[4-6]. 在代码生成领域, LLM 已被广泛用于补丁生成、代码补全、程序重构和缺陷修复等任务. 然而在真实工程场景下, 模型容易出现以下问题: 对缺陷类型的理解不够细粒度; 对领域特定规则的遵循度不足; 输出结果可能与预期修复逻辑存在偏差^[7,8].

Zhang 等人^[3]对学习型自动程序修复进行了系统综述, 总结了基于深度学习的方法在程序修复领域的进展. Xia 和 Zhang^[6]提出了基于对话的自动程序修复方法, 利用 ChatGPT 实现了高效的缺陷修复. Yin 等人^[13]提出了 ThinkRepair 方法, 通过自导向的自动程序修复策略提升了修复效果. 这些工作表明, LLM 在程序修复领域具有巨大潜力, 但如何有效利用领域上下文和规则信息仍然是一个开放问题.

2.3 RAG 与上下文增强

检索增强生成(RAG)方法通过在生成阶段动态引入外部知识, 能够显著增强模型对任务背景和领域知识的理解^[9,10]. RAG 的优势在于, 它可以把与当前缺陷相关的修复示例、规范说明或历史修复案例纳入提示词, 从而提高生成质量. Zhang 等人^[14]提出了基于检索增强的断言语词生成方法, 通过联合训练提升了检索和生成质量. 在代码修复领域, RELREPAIR^[15]通过检索相关函数签名和代码片段来增强自动程序修复. ReCode^[16]提出了细粒度的检索增强框架, 通过算法感知的检索策略和模块化双编码器架构实现了更准确的代码修复.

虽然 RAG 在理论上具备更强的知识注入能力, 但其实现依赖检索系统、向量索引、知识库维护以及额外的计算开销. 对于当前仓库所构建的实验环境而言, 若直接接入完整的 RAG 系统, 往往需要更大规模的知识库与更复杂的工程配置. 因此, 本文先从"规则上下文增强"这一更轻量、可控且易验证的方向入手, 研究在不引入检索模块的前提下, 规则提示是否能够显著提升修复效果.

3 HapRepair 系统架构与模块概览

本项目本身设计为一个完整的 RAG 增强 ArkTS 修复系统, 核心流水线包含以下模块:

(1) **缺陷检测**: 通过 HomeCheck/CodeLinter 规则移植为 ArkAnalyzer 静态分析, 输出待修复缺陷.

- (2) **缺陷上下文提取**: `get_surrounding_context.py` 从缺陷位置提取局部代码块和关联上下文, 并对重叠块进行合并.
- (3) **向量库构建**: `save_defects_to_database.py` 使用 `stella_en_1.5B_v5` 生成向量, 并将缺陷-修复对存入 Pinecone.
- (4) **Prompt 生成与 RAG 检索**: `get_prompt.py` 负责构造修复提示词, 结合 RAG 检索结果、规则说明、示例代码和输出格式要求.
- (5) **LLM 修复**: `fix.py` / `fix_projects.py` 执行多轮迭代修复, 支持 `gpt-diff`、`difflib`、`no-diff` 等策略.
- (6) **输出后处理与验证**: `output_handler.py` 负责从模型输出中提取补丁、修复 ArkTS 声明语法、检查功能等价.
- (7) **功能验证**: 通过 ArkAnalyzer 生成控制流图(control flow graph, CFG), 验证修复是否保持功能等价.
- (8) **跨文件依赖增强**: `cross_file_dependency.py` 用于分析项目级依赖、构建文件依赖图并提取跨文件上下文.

项目目录中还包含多个实验结果、数据集、图表和评估脚本, 说明该系统不仅支持单文件片段修复, 也具备向真实 OpenHarmony 工程扩展的设计能力.

本报告注意区分两类内容: 一方面, 是仓库当前实验中通过"规则上下文增强"获得的对比结果; 另一方面, 是项目整体设计中的 RAG+跨文件依赖增强架构. 当前实验基于 266 个缺陷对, 主要验证规则上下文、提示策略与模型语义理解的提升效果, 而非完整的 RAG 全链路性能.

4 方法设计

4.1 总体框架

HapRepair 项目的核心目标是利用大语言模型对 ArkTS 代码中的缺陷进行自动修复. 本文所研究的方法建立在现有仓库的修复流程之上, 整体框架包括以下几个阶段:

- (1) 缺陷样本读取与规则识别.
- (2) 规则上下文构造.
- (3) 基于提示词的修复生成.
- (4) 修复结果输出与对比评估.

源码仓库中与该流程相关的主要模块包括: `README.md` (项目总体说明与模块介绍)、`fix.py` (核心修复流程实现)、`get_prompt.py` (提示词生成与上下文构造逻辑)、`output_handler.py` (输出清洗、代码抽取与结果后处理)、`get_surrounding_context.py` (周边上下文提取)、`run_enhanced.py` (规则上下文增强实验入口)、`run_comparison.py` (基线与增强实验对比入口).

4.2 基线方法

基线方法采用"直接修复"策略. 对于每一个缺陷样本, 系统只向模型提供待修复代码片段, 而不提供任何额外的规则说明或领域上下文. 模型需要直接根据代码内容判断问题并生成修复补丁. 该方式的优点是实现简单, Prompt 负担较小, 但其缺点也很明显: 模型容易缺失领域规则感知, 尤其在复杂规则、结构化缺陷或上下文依赖较强的样本上, 修复效果不稳定^[7,8].

4.3 跨文件依赖分析集成到模型上下文

在增强方法中, 系统在输入模型前不仅构造规则上下文, 还将跨文件依赖分析结果显式整合到模型上下文中, 具体包括:

- (1) 缺陷规则名称.
- (2) 规则对应的自然语言描述.
- (3) 规则触发时的典型代码模式.
- (4) 与当前样本相关的代码上下文片段.
- (5) 依赖文件中的类型定义、接口声明、函数签名以及调用链信息.

传统的提示词方法通常只提供"修复这个代码"的指令, 而本文方法则将规则说明、局部上下文与跨文件依赖信息同时注入到修复提示中. 其核心思想是: 让模型在生成修复代码之前先"知道自己正在修复哪一类缺陷、该缺陷所在代码的局部语义, 以及该缺陷与其他文件中相关实体之间的依赖关系". 这种策略在经验上能够显著降低模型对规则语义与项目结构理解的偏差, 提高修复一致性.

在本项目中, 跨文件依赖上下文主要通过项目级依赖分析模块与提示词生成模块联合获得. 对于每个缺陷样本, 系统会先构建项目依赖图, 再根据缺陷位置提取相关依赖文件与符号信息, 并将其组织为结构化上下文片段嵌入到提示词中. 该策略本质上是对"依赖语义注入"的轻量实现, 具有较低的工程成本与较好的可迁移性.

4.4 评估方式

本文使用三个主要指标进行效果评估:

- (1) **修复率**: 表示模型成功修改缺陷代码样本的比例.
- (2) **精确匹配率**: 表示修复结果与期望修复代码完全一致的比例.
- (3) **规则级修复率**: 从规则维度分析不同缺陷类型下的修复表现.

此外, 为了分析方法的成本代价, 本文还统计了模型输入输出 Token 数量和估算费用, 用于判断上下文增强是否带来可接受的开销.

5 实验设计

5.1 实验数据

实验使用的数据集由 266 个缺陷对构成, 覆盖 36 种规则类型. 每个样本包含:

(1) 待修复的缺陷代码: 来自 `data/pairs/problem_code`.

(2) 期望修复代码: 来自 `data/pairs/repair_code`.

这一数据构成满足对比实验的基本要求, 因为其同时提供了"问题代码"和"正确修复代码", 便于定量评估模型修复结果与预期答案的一致程度.

5.2 实验模型与环境

实验环境如下:

(1) 操作系统: Microsoft Windows 11.

(2) Python 版本: 3.12.10.

(3) LLM 模型: DeepSeek-Chat.

(4) API 基址: <https://api.deepseek.com/v1>.

主要依赖包括: openai 2.33.0, pinecone 8.1.2, torch 2.11.0+cpu, transformers 5.7.0.

5.3 对比实验设置

本文采用了两组实验:

(1) **基线实验**: 直接修复, 不提供规则上下文.

(2) **增强实验**: 在提示词中加入缺陷规则名称、规则描述以及跨文件依赖上下文, 模拟"跨文件依赖分析集成到模型上下文"的增强策略.

需要强调的是, 本实验中的"增强"并非完整 RAG 系统, 而是基于结构化上下文提示的轻量增强策略. 该实验验证的是"依赖语义与规则语义联合引导"对 LLM 修复效果的影响, 并未直接评估完整的检索增强模块或更大规模工程级修复场景. 当前项目的整体架构中已设计了 RAG 与跨文件依赖模块, 但本次 266 个缺陷对对比主要集中于上下文集成的效果验证.

5.4 实验流程

实验流程可概括如下:

(1) 读取缺陷样本与对应期望修复文件.

(2) 对每个样本生成修复提示.

(3) 由模型输出修复后的代码.

(4) 对比修复结果与期望代码.

(5) 分别统计整体修复率、规则级修复率与 Token 消耗情况.

6 实验结果与分析

6.1 总体修复效果

实验结果表明, 规则上下文增强方法在整体修复任务上取得了稳定提升. 具体结果如表 1 所示.

版本	未修复数	修复数	修复率
Original(直接修复)	39	227	85.3%
Enhanced(规则上下文)	18	248	93.2%

表 1 总体修复效果对比

由表 1 可以看出, 增强方法相对于基线方法减少了 21 个未修复样本, 修复率从 85.3% 提升至 93.2%, 相对提升达到 7.9 个百分点. 这一结果说明, 在当前实验条件下, 规则上下文作为一种轻量型增强方式, 能够显著提升 LLM 对 ArkTS 缺陷的修复成功率.

6.2 规则级分析

进一步从规则维度观察实验结果, 可以看到大多数规则类型都受益于上下文增强. 表 2 给出部分代表性规则的对比结果.

规则	Original	Enhanced	提升
typed-array-check	40%	87%	+47%
use-local-var-to-replace-state-var	60%	100%	+40%
multiple-associations-state-var-check	27%	60%	+33%
hp-arkts-no-use-any-export-other	67%	100%	+33%
combine-same-arg-animateto	80%	100%	+20%
remove-redundant-state-var	80%	100%	+20%
constant-property-referencing-check-in-loops	82%	100%	+18%
load-on-demand	88%	100%	+12%
foreach-args-check	94%	100%	+6%
high-frequency-log-check	100%	100%	0%

表 2 部分规则修复率对比

从规则级分析可以看出, 针对结构性较强、语义较复杂的规则, 增强方法往往带来更明显的收益. 例如 typed-array-check 与 multiple-associations-state-var-check 这类规则的修复率提升非常显著. 这说明规则上下文增强对"需要模型理解规则背后语义"的任务更有效, 能够减少模型在修复时出现的误判和过度泛化.

6.3 Token 与成本分析

除了修复率之外, 本文还分析了上下文增强对模型调用成本的影响. 结果显示, 增强方法虽然引入了额外的规则描述上下文, 但总体 Token 增幅并不大. 具体如表 3 所示.

表 3 Token 使用与成本对比

版本	Input Tokens	Output Tokens	Total Tokens	预估费用(CNY)
Original	34,788	31,920	66,708	¥0.081
Enhanced	41,438	30,050	71,488	¥0.081
额外消耗	+6,650	-1,870	+4,780	+¥0.000

尽管增强方法增加了规则描述信息, 但总体模型调用成本几乎保持不变, 说明该方法在提升效果的同时, 并未显著增加费用负担. 这使得规则上下文增强方法具有较高的工程实用性, 尤其适合在中等规模缺陷修复任务中快速部署.

6.4 结果讨论

从实验结果看, 基于跨文件依赖上下文增强的方法取得了较为显著的收益, 主要原因可以归纳为三点:

- (1) 规则上下文帮助模型建立更明确的修复目标.
- (2) 跨文件依赖上下文降低了模型对缺陷所涉及项目结构关系的误判.
- (3) 结构化提示词提升了模型对代码模式与依赖语义的识别能力.

与此同时, 实验结果也显示了该方法的局限性. 对于某些本身已经具备较高修复成功率的规则, 例如 high-frequency-log-check, 增强方法并没有明显提升空间, 这说明方法收益与规则复杂度和规则歧义程度密切相关. 此外, 当前实验仍是"提示词增强"层面的验证, 尚未接入完整的 RAG 检索与跨文件依赖分析模块, 因此其性能上限仍有提升空间.

7 进一步讨论与展望

7.1 与完整 RAG 系统的关系

本文报告的实验结果是在未引入 RAG 的前提下获得的. 与完整的 RAG 系统相比, 当前方法更类似于"轻量级规则提示增强", 其优势在于实现简单、成本低、能够快速验证规则上下文是否有效. 对于后续构建完整的 RAG 修复系统来说, 这一实验可视为一个重要的前置验证: 若规则上下文本身都能显著提升修复效果, 那么在更大规模的知识库与检索系统支持下, 进一步提升性能的空间仍然相当可观^[9,10].

7.2 跨文件依赖分析的潜力

仓库中还包含 `cross_file_dependency.py` 等跨文件依赖分析模块. 该模块能够扫描工程级文件关系、构建依赖图并识别循环依赖, 并支持从依赖图中提取跨文件上下文信息. 项目文档显示, 该模块设计为纯 CPU 静态分析、零外部依赖, 可解析 ArkTS 的 `import/export`、装饰器、`@State/@Prop/@Link` 等语义, 并支持 BFS 提取依赖上下文.

对于真实 OpenHarmony 工程项目而言, 代码缺陷往往不只存在于单个文件中, 跨文件上下文常常决定修复是否正确. 当前项目还具备以下设计能力: 37 条性能/安全规则定义, 其中 15 条为上下文相关规则; 依赖图构建与循环依赖检测, 提升跨文件修复时的上下文可视化; 可选降级模式: 跨文件提取失败时静默跳过, 不影响单文件修复主流程; `get_prompt.py` 可将跨文件上下文以文本段落追加到修复 Prompt 中.

因此, 后续工作可以将规则上下文增强与跨文件依赖分析进行融合, 进一步提升修复质量与工程适配性. 当前本报告的实验结果为该系统设计提供了规则提示层面的验证基础.

7.3 未来工作方向

未来可以从以下几个方向继续推进:

- (1) 将当前基于规则提示的增强方法扩展为真正的 RAG 修复框架^[9,10,15,16].
- (2) 引入更丰富的修复示例与历史补丁库.
- (3) 构建面向 ArkTS 的领域知识图谱, 以提升模型对专业规则的理解.
- (4) 将修复结果与功能验证环节进一步结合, 提升修复后的代码可靠性.
- (5) 在更大规模项目数据集上进行验证, 尤其是真实 OpenHarmony 工程项目, 而不仅是代码片段数据集.

8 结论

本文围绕 HapRepair 项目, 系统分析了基于大语言模型的 ArkTS 缺陷修复方法, 并设计了一种规则上下文增强策略进行实验验证. 实验结果表明, 在 266 个缺陷对和 36 种规则类型的数据集上, 规则上下文增强方法相较于直接修复方法的修复率由 85.3% 提升至 93.2%, 提升了 7.9 个百分点, 显著减少了未修复样本数量. 与此同时, 规则级分析也表明, 大多数规则在增强方法下都获得了不同程度的收益.

需要说明的是, 本文所报告的优化效果是在未引入 RAG 的条件下得到的. 因此, 这一结果更准确地表述为"规则上下文增强方法相比直接修复方法具有显著优势", 而不是"完整 RAG 系统已经取得了最终最优效果". 这为后续构建基于检索增强的 HapRepair 完整版本提供了必要的实验基础, 也证明了在缺陷修复任务中, 结构化规则上下文是一种低成本、高收益的增强策略.

针对 ArkTS 这类具有明显规则特征的代码缺陷, 向大语言模型引入规则上下文能够有效提升修复准确率和稳定性, 为后续构建更完整的智能修复系统奠定了良好基础.

参考文献

- [1] Le Goues C, Pradel M, Roychoudhury A. Automated program repair. *Communications of the ACM*, 2019, 62(12): 56-65.

- [2] Liu K, Koyuncu A, Kim D, Bissyandé TF. TBar: Revisiting template-based automated program repair. In: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). New York: ACM, 2019. 31-42.
- [3] Zhang Q, Fang C, Ma Y, Sun W, Chen Z. A survey of learning-based automated program repair. ACM Transactions on Software Engineering and Methodology, 2023, 33(2): 55:1-55:69.
- [4] Feng Z, Guo D, Tang D, et al. CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020. 2020. 1536-1547.
- [5] Li Y, Wang S, Nguyen TN. DEAR: A novel deep learning-based approach for automated program repair. In: Proceedings of the 44th International Conference on Software Engineering (ICSE). New York: ACM, 2022. 511-523.
- [6] Xia C, Zhang L. Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. arXiv preprint arXiv:2304.00385, 2023.
- [7] Zhang Q, Zhang T, Zhai J, Fang C, Yu B, Sun W, Chen Z. A critical review of large language model on software engineering: An example from ChatGPT and automated program repair. arXiv preprint arXiv:2310.08879, 2023.
- [8] Fan Z, Gao X, Mirchev M, Roychoudhury A, Tan SH. Automated repair of programs from large language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023. 1469-1481.
- [9] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
- [10] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2023.
- [11] Ye H, Martinez M, Luo X, Zhang T, Monperrus M. SelfAPR: Self-supervised program repair with test execution diagnostics. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE). ACM, 2022. 92:1-92:13.
- [12] Ye H, Martinez M, Monperrus M. Neural program repair with execution-based backpropagation. In: Proceedings of the 44th International Conference on Software Engineering (ICSE). IEEE, 2022. 1506-1518.
- [13] Yin X, Ni C, Wang S, Li Z, Zeng L, Yang X. ThinkRepair: Self-directed automated program repair. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). New York: ACM, 2024. 1274-1286.
- [14] Zhang Q J, Fang C R, Zheng Y, Zhang Y X, Zhao Y, Huang R B, Zhou J Y, Yang Y, Zheng T, Chen Z Y. Improving deep assertion generation via fine-tuning retrieval-augmented

pre-trained language models[J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(7): 209.

[15] RELREPAIR: A novel approach that retrieves relevant code to enhance automated program repair. arXiv preprint arXiv:2509.16701, 2025.

[16] Zhang J, et al. ReCode: Improving LLM-based code repair with fine-grained retrieval-augmented generation. arXiv preprint arXiv:2509.02330, 2025.

[17] Dikici S, Bilgin TT. Advancements in automated program repair: A comprehensive review. Knowledge and Information Systems, 2025.

[18] Zhang Q, Fang C, Xie Y, Ma Y, Sun W, Yang Y, Chen Z. A systematic literature review on large language models for automated program repair. arXiv preprint arXiv:2405.01466, 2024.

[19] Jiang N, et al. A survey of LLM-based automated program repair. arXiv preprint arXiv:2506.23749, 2025.